

# AI-Followable Studio Architecture

Self-orchestrating blueprint for Codex, Claude Code, or a comparable coding agent

Design target: an AI-readable implementation document that can be executed incrementally by coding agents without inventing architecture on their own.

## 1. Executive summary

This architecture defines a two-system studio platform. The Mac mini hosts the always-on Studio Brain for business operations, while the Mac Pro or main production workstation hosts the audio pipeline. The platform is organized into five modules with strict approval boundaries, explicit tool wrappers, and a repository structure designed for agentic build-out. The intended outcome is a practical system that automates assistant-heavy overhead, accelerates audio preparation and QC, and keeps humans in control of creative, public, and financial decisions.

- Primary orchestration: OpenClaw.
- Workflow/event engine: n8n.
- Containment/runtime: Docker Compose.
- Local LLM serving: Ollama.
- Audio automation: SoundFlow, Pro Tools, and REAPER/ReaScript.
- Agent execution target: Codex or Claude Code consuming this document plus the repo task files.

## 2. Objectives and non-goals

| Area       | In scope                                                                                                                          | Out of scope                                                                                    |
|------------|-----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Operations | Lead intake, inbox triage, draft emails, reminders, social content, publishing queues, dashboarding, CRM sync, project follow-up. | Unreviewed cold outreach at scale, autonomous client commitments, autonomous financial actions. |
| Production | Session prep, routing, template application, candidate mixes, mastering variants, QC, revision handling.                          | A fully unsupervised final mix/master released without human approval.                          |
| Security   | Scoped permissions, queue-based approvals, containerized services, audit logs.                                                    | Granting the orchestrator blanket desktop, terminal, mailbox, or social platform access.        |

### 3. System topology

|                                         |                                                                                                                                                                                         |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mac mini - Studio Brain</b>          | Runs Docker services, OpenClaw, n8n, Ollama, vector/search helpers if needed, internal dashboard, approval queues, and business automation workflows. This machine should be always on. |
| <b>Mac Pro / Production Workstation</b> | Runs Pro Tools, REAPER, SoundFlow, plugin chains, analysis tools, rendering scripts, QC engine, and deliverables packaging. This machine performs the media-heavy workload.             |
| <b>Shared project storage</b>           | Holds watched folders, manifests, drafts, reports, exported deliverables, and handoff state. Can be local-first with optional backup/sync.                                              |
| <b>Human checkpoints</b>                | Review inbox drafts, review social drafts, approve booking language, approve candidate mix direction, approve final delivery and publishing.                                            |

### 4. Module definitions

| Module                        | Host                           | Inputs                                              | Outputs                                                         |
|-------------------------------|--------------------------------|-----------------------------------------------------|-----------------------------------------------------------------|
| 1. Session Intake & Prep      | Production workstation         | Stems, notes, references, project metadata          | Validated session, manifest, prep report                        |
| 2. AI Mix & Master            | Production workstation         | Prepared session, references, mix plan              | Candidate mixes, mastering variants, issue flags                |
| 3. QC & Delivery              | Production workstation         | Mixes, stems, masters, delivery spec                | Pass/fail report, package bundle, release artifacts             |
| 4. Studio Brain / Operations  | Mac mini                       | Forms, email, CRM, content calendar, project status | Draft replies, lead status, reminders, queued posts, dashboards |
| 5. Engineer Review & Revision | Production workstation + human | Mix candidates, notes, QC status                    | Approved changes, revision renders, client                      |

|  |  |  |                |
|--|--|--|----------------|
|  |  |  | review exports |
|--|--|--|----------------|

## 5. Control philosophy

- OpenClaw decides which workflow or tool wrapper to invoke next, but it does not directly own broad OS control.
- n8n handles event-driven automation, integrations, schedules, webhooks, and notification routing.
- Docker Compose contains the services and makes the environment reproducible for coding agents.
- Ollama hosts local models selected by role: a strong planner/drafter model and a smaller classifier/router model.
- Every sensitive action is modeled as either read-only, draft-only, queue-only, or explicitly approved automation.

## 6. Permission tiers

| Tier                                | Allowed actions                                                                        | Examples                                                            |
|-------------------------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Tier 1 - Read                       | Read approved inbox labels, project folders, CRM records, content calendar, manifests. | Read client intake forms; inspect “Needs Reply” mailbox label.      |
| Tier 2 - Draft                      | Write draft emails, draft social captions, draft reminders, draft project updates.     | Prepare a follow-up email without sending it.                       |
| Tier 3 - Queue                      | Place actions into an approval queue or scheduled queue.                               | Queue an Instagram caption for Maggie review.                       |
| Tier 4 - Narrow approved automation | Execute a very specific preapproved action.                                            | Send a standard reminder from a locked template after human opt-in. |

## 7. Reference repository layout

This section is intentionally precise so a coding agent can create the project skeleton without guessing.

- Repo root: studio-ai-platform/
- apps/studio-brain-ui/ - internal dashboard and approval queue UI.

- services/openclaw-orchestrator/ - orchestration prompts, tool wrappers, policy checks.
- services/n8n/ - exported workflows, credentials templates, webhook docs.
- services/ollama/ - model manifests, pull scripts, runtime notes.
- services/crm-api/ - lightweight internal lead/project service.
- services/content-pipeline/ - caption generation, asset packaging, publishing queue logic.
- services/audio-qc/ - loudness, peak, phase, render validation, report generation.
- services/project-state/ - canonical job state, manifests, revision history.
- workers/inbox-triage/
- workers/lead-intake/
- workers/social-drafting/
- workers/session-prep/
- workers/mix-planner/
- workers/revision-parser/
- workers/delivery-packager/
- infra/docker-compose.yml - full local stack.
- infra/env.example - required environment variables.
- docs/architecture/ - this document, ADRs, diagrams, prompt contracts.
- docs/runbooks/ - operator procedures and safety checklists.
- tasks/ - AI-followable task files for Codex or Claude Code.
- tests/ - unit, workflow, integration, and approval boundary tests.

## 8. Task-file pattern for coding agents

Every substantial implementation task should live in a dedicated markdown task file so the coding agent can work against explicit acceptance criteria. Each task file should include:

1. Purpose and scope.
2. Dependencies and required services.
3. Files expected to be created or modified.
4. Input and output contract.
5. Security constraints and forbidden shortcuts.
6. Acceptance tests.
7. Manual verification checklist.
8. Definition of done.

| Task file                      | Agent objective                                                                 |
|--------------------------------|---------------------------------------------------------------------------------|
| tasks/001-bootstrap-stack.md   | Create Docker Compose stack, env templates, health checks, and service readmes. |
| tasks/010-lead-intake.md       | Implement inbound lead parser, classification, and CRM write path.              |
| tasks/020-inbox-draft-queue.md | Implement safe email triage and draft queue                                     |

|                                    |                                                                                            |
|------------------------------------|--------------------------------------------------------------------------------------------|
|                                    | without send privileges.                                                                   |
| tasks/030-social-draft-pipeline.md | Implement content idea extraction, caption drafting, asset queueing, and review workflow.  |
| tasks/040-project-state-service.md | Implement canonical project/job state service and status transitions.                      |
| tasks/050-session-prep.md          | Implement manifest creation, file validation, routing template selection, and prep report. |
| tasks/060-audio-qc.md              | Implement loudness/peak/phase checks and export report generation.                         |
| tasks/070-revision-parser.md       | Translate human revision notes into controlled change instructions.                        |
| tasks/080-policy-guardrails.md     | Implement permission checks, audit logging, and approval queue enforcement.                |

## 9. Orchestration loop

- Trigger enters system via webhook, mailbox watcher, scheduler, file watcher, or operator action.
- n8n normalizes event into a job envelope.
- OpenClaw reads the envelope, current project state, and policy context.
- OpenClaw selects one tool wrapper or worker action.
- Worker performs deterministic work and writes structured output.
- Project-state service records status, artifacts, and next-action recommendations.
- If human review is required, the job moves to an approval queue; otherwise the next bounded step runs.
- Completion writes an audit entry and updates dashboards.

## 10. Canonical job envelope

Recommended JSON shape for events passed between orchestrator, workers, and queues.

```
{
  "job_id": "uuid",
  "module": "studio_brain|session_prep|mix|qc|delivery",
  "action": "lead_intake|draft_email|generate_mix_plan|run_qc",
  "project_id": "optional-project-id",
```

```

"priority": "low|normal|high",
"inputs": {"...": "..."},
"required_permissions": ["read", "draft"],
"approval_required": true,
"artifacts": [],
"created_at": "ISO-8601",
"requested_by": "system|human|workflow"
}

```

## 11. Model allocation

| Role                          | Recommended local model profile | Reason                                                                       |
|-------------------------------|---------------------------------|------------------------------------------------------------------------------|
| Planner / drafter             | Qwen-adjacent instruct model    | Strong structured writing, decent reasoning, good local viability.           |
| Classifier / router           | Smaller fast local model        | Fast labeling, triage, urgency scoring, and routing decisions.               |
| Embeddings / retrieval        | Light embedding model           | Useful for project memory, templates, and CRM retrieval.                     |
| Audio-adjacent language tasks | Use planner sparingly           | Best for notes, plans, and summaries rather than sample-level DSP decisions. |

## 12. Module 4 - Studio Brain implementation details

### Lead intake workflow

- Accept forms, DMs converted to structured intake, or email inquiries.
- Normalize fields: artist name, service requested, timeline, budget signal, deliverables, references, urgency.
- Score fit and urgency using deterministic rules plus a lightweight classifier.
- Draft a first response in Maggie's tone and create a follow-up timer.
- Write to CRM and dashboard; do not send without approval unless workflow is explicitly trusted.

### Inbox triage workflow

- Only inspect approved labels or folders.
- Classify message type: lead, revision request, scheduling, payment, admin, noise.

- Draft responses where safe; escalate ambiguous or sensitive threads.
- Never delete, archive destructively, or auto-send unless separately authorized.

Social/content workflow

- Generate post ideas from project milestones, testimonials, before/after moments, release dates, and studio updates.
- Prepare platform-specific captions, short-form variants, hashtags, and asset manifests.
- Queue for review and schedule via connector only after approval.

13. Audio-side implementation details

- Session Prep worker validates stems, length alignment, sample rate, bit depth, and naming before building a template session.
- Mix Planner worker writes a structured plan: focal elements, low-end strategy, vocal stance, bus chain family, and risk flags.
- Execution wrappers call SoundFlow macros or REAPER scripts rather than freeform UI clicking.
- QC worker checks LUFs, true peak, clipping, phase/correlation, mono fold-down behavior, render length, and package completeness.
- Revision Parser converts notes like “vocal up in chorus” into parameterized changes instead of vague instructions.

14. Safety and audit requirements

| Requirement      | Implementation expectation                                                                               |
|------------------|----------------------------------------------------------------------------------------------------------|
| Audit trail      | Every automated action and approval event is logged with timestamp, actor, and artifact references.      |
| Prompt isolation | System prompts, user prompts, and tool outputs should be stored separately for debugging and evaluation. |
| Secrets hygiene  | Use env vars or a secrets manager; never hardcode tokens in task files or prompts.                       |
| Approval gating  | Sensitive tasks must fail closed and enter review if policy context is missing.                          |
| Reproducibility  | All services should start from Docker Compose with documented volumes and health checks.                 |

15. Build order for a coding agent

9. Bootstrap repo skeleton, Docker Compose, env templates, and service health checks.

10. Implement project-state service and approval queue UI.
11. Implement lead intake and inbox drafting workflows on the Mac mini.
12. Implement social draft pipeline and scheduled reminder engine.
13. Implement audio session prep worker and manifest generator.
14. Implement audio QC worker and reporting.
15. Implement mix-planner worker and revision parser.
16. Add policy guardrails, audit logging, and evaluation tests across all modules.
17. Run end-to-end scenario tests covering lead -> booked project -> delivery -> follow-up.

## 16. Acceptance criteria

- A coding agent can stand up the full local stack from the repo readme and env example.
- Lead intake creates a structured record, draft reply, and reminder without sending anything automatically.
- Inbox triage produces a reviewable queue with no destructive actions.
- A sample audio project can move through session prep, QC, and revision parsing with reproducible outputs.
- Every sensitive action is blocked or queued when approval state is missing.
- Operator documentation is sufficient for Maggie to understand where the human checkpoints live.

## 17. Suggested next artifacts

- A repository bootstrap package with placeholder services and task files.
- An ADR set covering OpenClaw scope boundaries, approval policies, and model selection.
- Sample n8n workflows for lead intake, inbox draft queue, and post scheduling.
- A dashboard wireframe for the approval queue and project status view.